

6.7 В недрах BuildContext

Шаг 1

Как уже говорилось ранее, во Flutter **все является виджетом**. Из кода приложения формируется дерево виджетов, которое в runtime преобразуется в дерево элементов. А непосредственно на экране, пользователь видит отрисованное дерево из `RenderObject`, сформированное из дерева элементов. При этом, не каждый элемент имеет свой `RenderObject`, используя в таком случае его представление из родительского элемента. Поэтому дерево рендеринга получается значительно миниатюрнее предыдущих двух деревьев. Но на самом деле, во **Flutter нет дерева виджетов!!!** Это понятие используется для удобства восприятия, так как программист в основном оперирует виджетами и не задумывается, что у них под «капотом». Вся основная работа фреймворка завязана на элементах, дереве элементов и рендеринга.

У каждого виджета имеется метод `createElement()`, который вызывается Flutter и возвращает его элемент, использующий данный виджет для своей конфигурации и отвечающий за его управление расположением в дереве. Жизненный цикл таких элементов можно представить следующим образом:

- Flutter создает элемент на основе конфигурации виджета, вызывая его метод `Widget.createElement`.
- Для добавления созданного элемента в дерево, относительно его родительского элемента, Flutter использует метод `Element.mount`, который отвечает за создание любых дочерних элементов виджетов и вызов `Element.attachRenderObject`, когда необходимо прикрепить к дереву рендеринга все связанные с ним объекты.
- Элемент становится «активным» и может появиться на экране.
- Если в какой-то момент времени родитель решит изменить виджет, используемый для настройки этого элемента, у нового виджета Flutter вызовет `Element.update`. Этот виджет всегда будет иметь то же значение `runtimeType` и ключа, что и старый виджет. Для того, чтобы родитель мог изменить `runtimeType` или ключ виджета в данном месте дерева, он должен размонтировать этот элемент и создать в этом месте дерева новый виджет.
- Если в какой-то момент времени родительский элемент решит удалить этот дочерний элемент (или промежуточного потомка) из дерева, он вызовет метод `Element.deactivateChild`. В результате произойдет удаление из дерева рендеринга объекта рендеринга этого элемента, после чего сам элемент будет добавлен в список неактивных элементов, тем самым заставив Flutter вызвать у этого элемента метод `Element.deactivate`.
- Элемент становится «неактивным» и перестает отображаться на экране. В таком состоянии он может оставаться до конца текущего кадра анимации, по завершению которого все неактивные элементы будут размонтированы.
- В том случае, если элемент будет повторно включен в дерево (например, элемент или один из его предков имеет глобальный ключ, который используется повторно), Flutter удалит его из списка неактивных элементов, сделав снова «активным», вызвав метод `Element.activate`. После чего повторно прикрепит объект рендеринга элемента к дереву рендеринга.
- Если элемент до конца текущего кадра анимации не включается повторно в дерево, Flutter вызовет у него метод `Element.unmount`.
- Элемент считается «устаревшим» и в будущем не будет включен в дерево.

Каждый элемент реализует интерфейс абстрактного класса `BuildContext`, приводясь к интерфейсу которого и передается в метод `StatelessWidget.build` или `State.build` в качестве переменной `context`:

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ),
        useMaterial3: true,
      ),
      home: const HomePage(),
    );
  }
}
```



Передаваемый в метод `build` контекст относится только к конкретному виджету и считается родительским по отношению к виджету, возвращаемому методом `StatelessWidget.build` или `State.build`. В случае `StatelessWidget` доступ к контексту имеется только в методе `build`, либо когда он явно передается из него в другие методы виджета или функции модуля. У класса `State` доступ к контексту имеется как в методе `build`, через входной аргумент `context`, так и из любого места, поскольку связывается с его одноименным полем.

Но зачем еще нужен `BuildContext` помимо участия в формировании дерева элементов и хранения информации о местоположении в нем текущего виджета? Все дело в том, что, используя контекст, мы можем обращаться к родительским виджетам, находящимся на более высоких уровнях дерева, получая доступ к их свойствам и методам. Также `BuildContext` можно использовать для передачи сообщения по дереву виджетов от родительского к дочернему и обратно (для этого используется `InheritedWidget`, который рассмотрим в следующем разделе).

Класс `BuildContext` имеет следующие уникальные свойства [7]:

- `debugDoingBuild` → `bool` – возвращает `true` или `false` в зависимости от того, обновляется ли в данный момент виджет или дерево рендеринга.
- `mounted` → `bool` – возвращает `true` или `false` в зависимости от того, установлен ли виджет, с которым связан данный контекст, в дерево элементов.
- `owner` → `BuildOwner?` – возвращает `BuildOwner` текущего контекста, который отвечает за управление конвейером рендеринга данного контекста.
- `size` → `Size?` – возвращает реальный размер виджета, отображаемого на экране. К данному свойству следует обращаться только после того, как виджет был отрисован! Да и в принципе, не стоит им слишком злоупотреблять.
- `widget` → `Widget` – возвращает текущую конфигурацию элемента, связанного с конкретным `BuildContext`.

Помимо свойств, он предоставляет разработчику следующие методы для работы с ним:

- `dependOnInheritedElement(InheritedElement ancestor, {Object? aspect})` → `InheritedWidget` – используется для связывания текущего контекста сборки с предком, что позволяет производить его перестройку при изменении виджета предка.
- `dependOnInheritedWidgetOfExactType<T extends InheritedWidget>({Object? aspect})` → `T?` – возвращает ближайший виджет заданного типа `T` и создает зависимость от него. Если виджет не найдет, вернется `null`.
- `describeElement(String name, {DiagnosticsTreeStyle style = DiagnosticsTreeStyle.errorProperty})` → `DiagnosticsNode` – возвращает описание элемента, связанного с текущим `BuildContext`.
- `describeMissingAncestor({required Type expectedAncestorType})` → `List<DiagnosticsNode>` – используется для добавления описания конкретного типа виджета, отсутствующего в дереве предков текущего `BuildContext`.
- `describeOwnershipChain(String name)` → `DiagnosticsNode` – используется для добавления описания цепочки владения

конкретного элемента в отчет об ошибке.

- `describeWidget(String name, {DiagnosticsTreeStyle style = DiagnosticsTreeStyle.errorProperty}) → DiagnosticsNode` – возвращает описание виджета, связанного с текущим `BuildContext`.
- `dispatchNotification(Notification notification) → void` – данный метод запускает рассылку передаваемого уведомления в текущем контексте сборки.
- `findAncestorRenderObjectOfType<T extends RenderObject>() → T?` – возвращает `RenderObject` ближайшего предка виджета `RenderObjectWidget`, являющегося экземпляром заданного типа `T`. Если объект не найден, вернется `null`.
- `findAncestorStateOfType<T extends State<StatefulWidget>>() → T?` – возвращает `State` ближайшего предка виджета `StatefulWidget`, являющегося экземпляром заданного типа `T`. Если объект не найден, вернется `null`.
- `findAncestorWidgetOfExactType<T extends Widget>() → T?` – возвращает ближайший родительский виджет заданного типа `T`, являющийся типом конкретного подкласса `Widget`. Если объект не найден, вернется `null`.
- `findRenderObject() → RenderObject?` – возвращает `RenderObject` для текущего виджета. Если элемент виджета не имеет `RenderObjectWidget` (то есть не отображается на экране), то вернется объект рендеринга первого потомка `RenderObjectWidget`.
- `findRootAncestorStateOfType<T extends State<StatefulWidget>>() → T?` – возвращает `State` самого дальнего предка виджета `StatefulWidget`, являющегося экземпляром заданного типа `T`. Если объект не найден, вернется `null`.
- `getElementForInheritedWidgetOfExactType<T extends InheritedWidget>() → InheritedElement?` – возвращает элемент ближайшего виджета заданного типа `T`, который является производным классом от `InheritedWidget`.
- `getInheritedWidgetOfExactType<T extends InheritedWidget>() → T?` – возвращает ближайший виджет заданного типа `T`, который является производным классом от `InheritedWidget`. Если подходящий виджет не найден, вернется `null`.
- `visitAncestorElements(ConditionalElementVisitor visitor) → void` – данный метод используется для организации обхода по цепочке родительских элементов, начиная с элемента текущего `BuildContext`.
- `visitChildElements(ElementVisitor visitor) → void` – данный метод используется для организации обхода по цепочке дочерних элементов текущего виджета.

Ряд виджетов (`Theme`, `Scaffold` и т.д.), для их поиска в дереве элементов, предоставляют статический метод `of` на вход которого подается экземпляр `BuildContext`. Также, контекст используется для организации навигации между экранами:

```
// ex_build_context1/lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ),
        useMaterial3: true,
      ),
      home: const HomePage(),
    );
  }
}

class HomePage extends StatelessWidget {
  const HomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Home Page'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            const Text(
              'Current Theme Primary Color:',
              style: TextStyle(fontSize: 20),
            ),
            const SizedBox(height: 10),
            // Использование BuildContext для доступа к теме
            Container(
              width: 100,
              height: 100,
              color: Theme.of(context).primaryColor,
            ),
            const SizedBox(height: 20),
            ElevatedButton(
              child: const Text('Go to Details'),
              onPressed: () {
                // Использование BuildContext для навигации
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (context) => const DetailsPage(),
                  ),
                );
              },
            ),
          ],
        ),
      ),
    );
  }
}
```

```
}

class DetailsPage extends StatelessWidget {
  const DetailsPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Details Page'),
      ),
      body: Center(
        child: ElevatedButton(
          child: const Text('Go Back'),
          onPressed: () {
            // Использование BuildContext для навигации назад
            Navigator.pop(context);
          },
        ),
      ),
    );
  }
}
```

Шаг 2

Поскольку экземпляр `BuildContext` – это элемент, мы можем удариться в «уличную магию», вернувшись к примеру из `StatelessWidget`, где нажатие на кнопку не приводило ни к каким изменениям счетчика на экране и внеся изменение всего лишь в пару строк кода, сделать перерисовку неизменяемого виджета. **Пример ниже носит демонстративный характер и за его перенос в реальный проект вас могут сильно покарать более старшие коллеги, т.к. для таких задач лучше использовать `StatefulWidget`, а не `BuildContext`:**

```
// ex_build_context2/lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    // без изменения
  }
}

class MyHomePage extends StatelessWidget {
  int counter;
  String title;

  MyHomePage({
    // без изменения
  });

  void _incrementCounter(BuildContext context) {
    counter++;
    // вызываем перерисовку элемента, запросив его обновление
    (context as Element).markNeedsBuild();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: // без изменения,
      body: // без изменения,
      floatingActionButton: FloatingActionButton(
        onPressed: () => _incrementCounter(context),
        tooltip: 'Increment',
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Конечно, на более низком уровне метод `setState` у `State` использует аналогичный способ для запуска перерисовки при изменении состояния, но это не значит, что такой велосипедо-костыльный способ надо использовать для `StatelessWidget`. Лучшей политикой будет придерживаться тех концепций, за которые эти виджеты отвечают, а не превращать проект в минное поле.

Шаг 3

Для лучшего понимания зацепления контекста в методе `build` и поиска с его помощью нужных родительских виджетов, давайте реализуем несколько примеров как с `StatelessWidget`, так и с `StatefulWidget`. В этом нам поможет простое приложение со следующим графическим пользовательским интерфейсом (GUI):

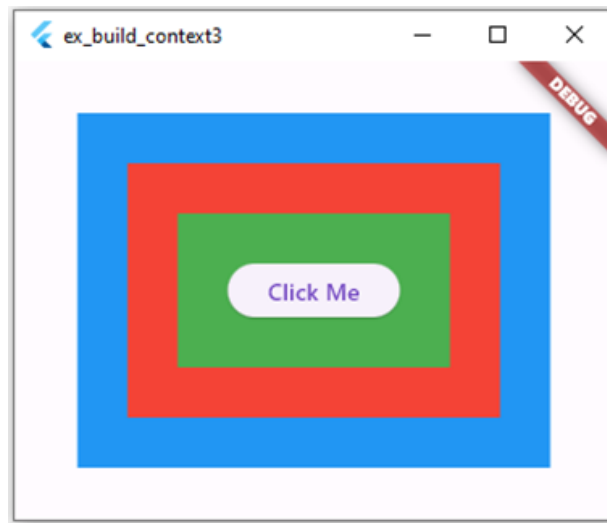


Рис. 1.39. – Пример GUI реализуемого приложения

По нажатию на центральную кнопку в приложении будет осуществляться поиск первого родительского виджета (цветного контейнера) `MyColorBox`, относительно зацепленного контекста. Начнем с `StatelessWidget` и в первом варианте осуществим верстку и объявим кнопку в методе `build` корневого виджета:

```
// ex_build_context3/lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ),
        useMaterial3: true,
      ),
      home: Scaffold(
        body: Center(
          child: MyColorBox(
            color: Colors.blue,
            colorName: 'Blue',
            child: MyColorBox(
              color: Colors.red,
              colorName: 'Red',
              child: MyColorBox(
                color: Colors.green,
                colorName: 'Green',
                child: ElevatedButton(
                  onPressed: () {
                    debugPrint(
                      MyColorBox.ofColor(context)?.toString(),
                    );
                    debugPrint(
                      MyColorBox.of(context)?.toString(),
                    );
                  },
                  child: const Text('Click Me'),
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}

class MyColorBox extends StatelessWidget {
  static const padding = 30.0; // внутренний отступ
  final String colorName; // название цвета
  final Color color; // цвет
  final Widget? child; // дочерний виджет

  const MyColorBox({
    super.key,
    required this.colorName,
    required this.color,
    this.child,
  });

  @override
  Widget build(BuildContext context) {
    return Container(
```



```
padding: const EdgeInsets.all(
  MyColorBox.padding,
),
color: color,
child: child,
);
}

static Color? ofColor(BuildContext context) {
  // возвращает цвет первого найденного родительского
  // виджета или null
  return
    context.findAncestorWidgetOfExactType<MyColorBox>()?.color;
}

static MyColorBox? of(BuildContext context) {
  // возвращает первый найденный родительский виджет или null
  return context.findAncestorWidgetOfExactType<MyColorBox>();
}
}
```

В текущем примере, результатом нажатия на кнопку будет вывод в терминал значения null. Это связано с тем, что мы захватили контекст элемента, который ничего не знает о своих дочерних виджетах, а поиск используемыми методами всегда осуществляется по родительским элементам текущего контекста. Чтобы исправить эту ситуацию можно вынести кнопку в отдельный пользовательский виджет MyButton, который поместим в зеленый контейнер:

```
// ex_build_context4/lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: // без изменений,
      home: const Scaffold(
        body: Center(
          child: MyColorBox(
            color: Colors.blue,
            colorName: 'Blue',
            child: MyColorBox(
              color: Colors.red,
              colorName: 'Red',
              child: MyColorBox(
                color: Colors.green,
                colorName: 'Green',
                child: MyButton(),
              ),
            ),
          ),
        ),
      ),
    );
  }
}

class MyButton extends StatelessWidget {
  const MyButton({super.key});

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () {
        debugPrint(
          MyColorBox.ofColor(context)?.toString(),
        );
        debugPrint(
          MyColorBox.of(context)?.colorName,
        );
      },
      child: const Text('Click Me'),
    );
  }
}

class MyColorBox extends StatelessWidget {
  // без изменений
}
```

В данном случае, при нажатии кнопки, мы осуществляем поиск ближайшего родительского виджета `MyColorBox` в рамках контекста метода `build` виджета `MyButton`, который был указан в качестве дочернего для контейнера зеленого цвета. В следствие чего в терминал будут выведены следующие строки:

```
flutter: MaterialColor(primary value: Color(0xff4caf50))
flutter: Green
```

Если поменять верстку и расположим кнопку на одном уровне с зеленым контейнером, как это показано на следующем рисунке, с учетом изменений в дереве элемента и захватываемого контекста виджетом MyButton в терминал будет выводиться:

```
flutter: MaterialColor(primary value: Color(0xffff44336))  
flutter: Red
```

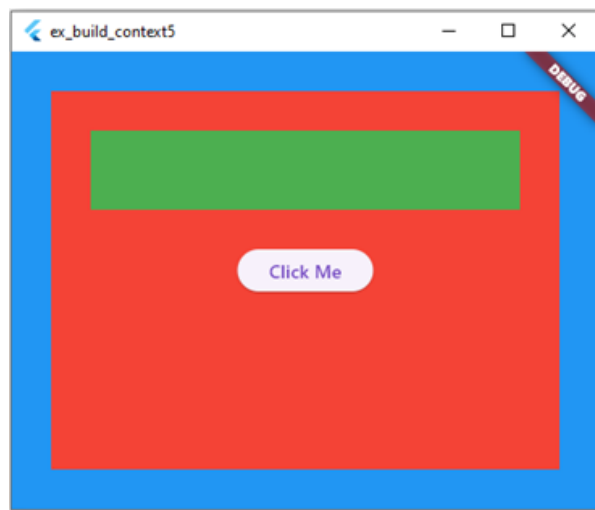


Рис. 1.40. – Пример GUI реализуемого приложения

```
// ex_build_context5/lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: // без изменений,
      home: const Scaffold(
        body: Center(
          child: MyColorBox(
            color: Colors.blue,
            colorName: 'Blue',
            child: MyColorBox(
              color: Colors.red,
              colorName: 'Red',
              child: Column(
                children: [
                  MyColorBox(
                    color: Colors.green,
                    colorName: 'Green',
                  ),
                  SizedBox(height: 30),
                  MyButton(),
                ],
              ),
            ),
          ),
        ),
      ),
    );
  }
}

class MyButton extends StatelessWidget {
  // без изменений
}

class MyColorBox extends StatelessWidget {
  // без изменений
}
```

Шаг 4

Теперь перейдем к примерам со `StatefulWidget` и по нажатию кнопки будем менять цвет контейнера. В первом случае, так как `BuildContext` предоставляет подходящие методы, это будет самый первый и последний контейнер. При вызове `findAncestorStateOfType<T>` вернется самый первый родительский виджет (`State`) относительно текущего контекста, который будет найден в дереве. А `findRootAncestorStateOfType<T>` вернет последний виджет заданного типа в дереве. Поскольку эти методы запускают обход дерева и имеют линейную временную сложность $O(n)$, при большом количестве элементов, их использование для обращения к конкретному виджету для вызова его методов – не самая лучшая идея. Метод же `findRootAncestorStateOfType` так вообще, осуществляет проход до корня дерева, после чего возвращает последний найденный `State` заданного типа.

Для начала перепишем существующий `StatelessWidget` `MyColorBox` на `StatefulWidget`, добавив в него метод, осуществляющий

замену цвета:

```
// ex_build_context6/lib/main.dart
class MyColorBox extends StatefulWidget {
  static const padding = 30.0;
  final Color color;
  final Widget? child;

  const MyColorBox({
    super.key,
    required this.color,
    this.child,
  });

  @override
  State<MyColorBox> createState() => _MyColorBoxState();
}

class _MyColorBoxState extends State<MyColorBox> {
  Color _color = Colors.transparent;
  Widget? _child;

  @override
  void initState() {
    super.initState();
    setState(() {
      _color = widget.color;
      _child = widget.child;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(
        MyColorBox.padding,
      ),
      color: _color,
      child: _child,
    );
  }

  void changeColor(Color color) {
    setState(() {
      _color = color;
    });
  }
}
```

Класс виджета MyButton тоже претерпит изменения. Добавим ему несколько аргументов, чтобы можно было передавать анонимную функцию, которая будет вызываться по нажатию и текст, отображаемый на кнопке:

```
// ex_build_context6/lib/main.dart
class MyButton extends StatelessWidget {
  final void Function(BuildContext context) onPressed;
  final String text;
  const MyButton({
    super.key,
    required this.onPressed,
    required this.text,
  });

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () {
        onPressed(context);
      },
      child: Text(text),
    );
  }
}
```

Функции, передаваемые в конструктор MyButton можно объявить несколькими способами. При указании аргумента конструктора передать ему анонимную функцию, объявленную в самом конструкторе, либо объявить ее в другом месте в качестве функции верхнего уровня (или метода класса), после чего передать имя функции аргументу onPressed класса MyButton. Для большей наглядности воспользуемся вторым подходом и на верхнем уровне модуля объявим несколько функций:

```
// ex_build_context6/lib/main.dart
// Функция для поиска самого первого родительского MyColorBoxState в
// дереве виджетов (относительно контекста) для изменения цвета.
void firstParentChangeColor(BuildContext context) {
  final myColorBox =
    context.findAncestorStateOfType<_MyColorBoxState>();
  if (myColorBox?._color == Colors.green) {
    myColorBox?.changeColor(Colors.grey);
  } else {
    myColorBox?.changeColor(Colors.green);
  }
}

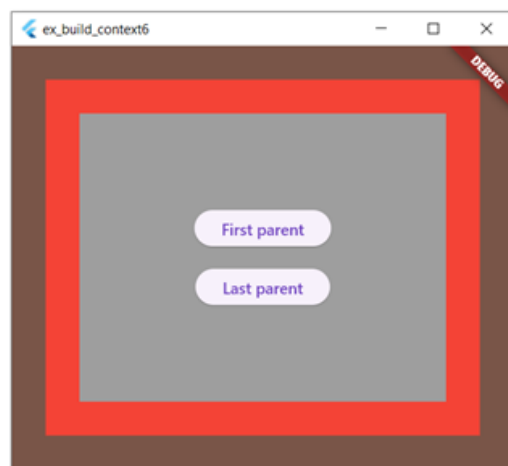
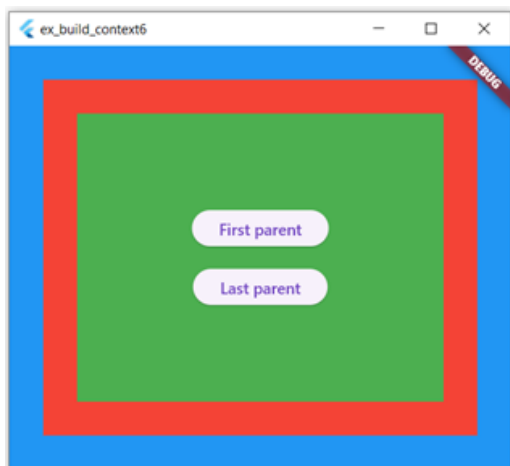
// Функция для поиска самого верхнего родительского MyColorBoxState в
// дереве виджетов для изменения цвета.
void lastParentChangeColor(BuildContext context) {
  final myColorBox =
    context.findRootAncestorStateOfType<_MyColorBoxState>();
  if (myColorBox?._color == Colors.blue) {
    myColorBox?.changeColor(Colors.brown);
  } else {
    myColorBox?.changeColor(Colors.blue);
  }
}
```

Далее изменим код виджета MyApp, добавив в последний контейнер две кнопки вместо одной. Первая будет отвечать за изменение цвета ближайшего родителя, а вторая – самого дальнего:

```
// ex_build_context6/lib/main.dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ),
        useMaterial3: true,
      ),
      home: const Scaffold(
        body: Center(
          child: MyColorBox(
            color: Colors.blue,
            child: MyColorBox(
              color: Colors.red,
              child: MyColorBox(
                color: Colors.green,
                child: Center(
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.min,
                    children: [
                      MyButton(
                        text: 'First parent',
                        onPressed: firstParentChangeColor,
                      ),
                      SizedBox(height: 20),
                      MyButton(
                        text: 'Last parent',
                        onPressed: lastParentChangeColor,
                      ),
                    ],
                  ),
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

Теперь можно запустить приложение и поиграться с изменением цвета самого верхнего и последнего контейнера, нажимая на соответствующие кнопки:



Шаг 5

А как же быть со средним цветным контейнером? Для изменения его цвета можно применить несколько подходов. Один из них мы рассмотрим в следующем разделе, а два здесь. Так как известно, что средний контейнер является родителем по отношению к третьему и они реализованы посредством `StatefulWidget`, то по нажатию на кнопку сначала получается `State` первого родителя заданного типа и уже используя его контекст, запросить `State` центрального цветного контейнера, т.е. его родителя. Для воплощения в жизнь такого «финта ушами», объявим следующую функцию:

```
// ex_build_context7/lib/main.dart
// Функция для поиска второго родительского MyColorBoxState в
// дереве виджетов для изменения цвета.
void secondParentChangeColor(BuildContext context) {
  final myColorBox =
    context.findAncestorStateOfType<_MyColorBoxState>();
  final secondColorBox =
    myColorBox?.context.findAncestorStateOfType<_MyColorBoxState>();
  if (secondColorBox?._color == Colors.red) {
    secondColorBox?.changeColor(Colors.grey);
  } else {
    secondColorBox?.changeColor(Colors.red);
  }
}
```

После чего внесем изменения в конфигурацию виджета `MyApp`, оставив одну кнопку:


```
// ex_build_context7/lib/main.dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ),
        useMaterial3: true,
      ),
      home: const Scaffold(
        body: Center(
          child: MyColorBox(
            color: Colors.blue,
            child: MyColorBox(
              color: Colors.red,
              child: MyColorBox(
                color: Colors.green,
                child: Center(
                  child: MyButton(
                    text: 'Second parent',
                    onPressed: secondParentChangeColor,
                  ),
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

Теперь по нажатию на кнопку будет осуществляться изменение цвета центрального контейнера:

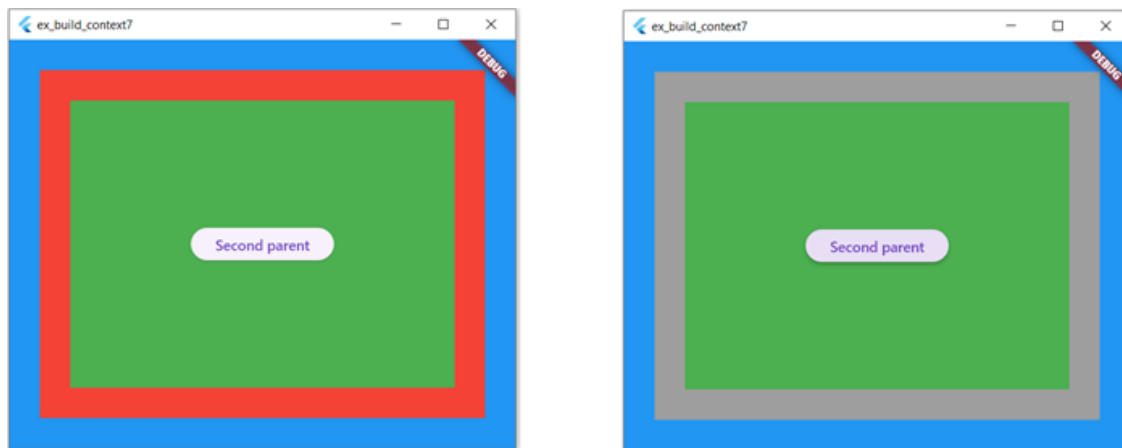


Рис. 1.42. – Пример работы приложения

Следующий способ заключается в использовании глобального ключа, который можно объявить на верхнем уровне, передать в конструктор среднего цветного контейнера и в функции, вызываемой по нажатию на кнопку, получить по данному ключу State необходимого виджета, после чего вызвать метод изменения его цвета:

```
// ex_build_context8/lib/main.dart
final myKey = GlobalKey<_MyColorBoxState>();

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ),
        useMaterial3: true,
      ),
      home: Scaffold(
        body: Center(
          child: MyColorBox(
            color: Colors.blue,
            child: MyColorBox(
              key: myKey,
              color: Colors.red,
              child: const MyColorBox(
                color: Colors.green,
                child: Center(
                  child: MyButton(
                    text: 'Second parent',
                    onPressed: secondParentChangeColor,
                  ),
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}

// Функция для поиска второго родительского MyColorBoxState в
// дереве виджетов для изменения цвета.
void secondParentChangeColor(BuildContext context) {
  final myColorBox = myKey.currentState;
  if (myColorBox?._color == Colors.red) {
    myColorBox?.changeColor(Colors.grey);
  } else {
    myColorBox?.changeColor(Colors.red);
  }
}
```

Шаг 6

Более подробно про то, зачем нужны ключи и как их использовать мы рассмотрим в одном из следующих разделов. А пока давайте подведем итоги:

1. BuildContext предоставляет разработчику достаточно удобный интерфейс для поиска необходимых виджетов в дереве, абстрагируя собой экземпляр элемента виджета, который создается Flutter. Он позволяет организовать передачу сообщений как от дочернего виджета в родительский, так и наоборот, но нужно быть внимательными при использовании ряда методов, так как они имеют линейную сложность. Поэтому при наличии в пользовательском интерфейсе огромного количества виджетов лучше использовать другие механизмы для вызова метода родительского

виджета из дочернего.

2. Передаваемый в метод `build` контекст относится только к конкретному виджету и считается родительским по отношению к виджету, возвращаемому методом `StatelessWidget.build` или `State.build`. Это следует учитывать при обращении через контекст к родительскому виджету, так как может возникнуть ситуация, что по вашему мнению один виджет является родительским, а другой дочерним, но они объявлены на одном уровне и метод поиска захватывает контекст более высокого уровня. Один из наиболее предпочтительных способов избежать такого поведения – объявить отдельный виджет и работать на уровне его контекста. Другой – использовать виджет `Builder`, который позволяет создать для текущего элемента виджета в дереве, элемент более низкого уровня. И если для ряда виджетов, таких как `ListView` с его именованным конструктором `builder` это нормальная практика, то виджет `Builder` способен внести сумятицу в восприятие кода приложения и трактовку дерева элементов текущей страницы GUI.
3. К свойству `BuildContext.size` необходимо обращаться с особой осторожностью, т.к. оно будет доступно только после отрисовки виджета.
4. Если вам дорого свое психологическое и физическое здоровье, то не следует пользоваться «уличной магией» элементов и превращать `StatelessWidget` в `StatefulWidget`.

Шаг 7

Что из перечисленного наиболее точно описывает `BuildContext` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1256675/step/7>

Шаг 8

Как из `StatelessWidget` можно сделать `StatefulWidget` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1256675/step/8>

Шаг 9

Что делает метод `context.findAncestorStateOfType<T>()` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1256675/step/9>

Шаг 10

Что делает метод `context.findRootAncestorStateOfType<T>()` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1256675/step/10>

Шаг 11

Какая временная сложность работы методов `findAncestorStateOfType` и `findRootAncestorStateOfType` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1256675/step/11>

Шаг 12

Сопоставьте методы `BuildContext` с их назначение

Чтобы решить это задание откройте <https://stepik.org/lesson/1256675/step/12>

Шаг 13

Сопоставьте методы `BuildContext` с их назначение

Чтобы решить это задание откройте <https://stepik.org/lesson/1256675/step/13>

Шаг 14

Какие из приведенных утверждений верны относительно `BuildContext` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1256675/step/14>

Шаг 15

Какие утверждения про методы поиска предков через `BuildContext` верны?

Чтобы решить это задание откройте <https://stepik.org/lesson/1256675/step/15>

Шаг 16

Какие утверждения верны про алгоритм работы `findRootAncestorStateOfType<T>()` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1256675/step/16>